

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD PERUANA LOS ANDES



UPLA

TEMA: Resolución de la actividad 2 Semana 10

CARRERA: Ingeniería de sistemas y comunicación

FACULTAD: Ingeniería

ESTUDIANTE: Cuba Villanueva Javier Amilcar

DOCENTE: Raul Enrique Fernández Bejarano

Huancayo 2026

Problema 1

Enunciado:

Modificar la base de datos **GradosTitulos** agregando un nuevo grupo de archivos denominado **IdxGroup** y asociarle un archivo físico secundario (.ndf). Este grupo almacenará los índices no agrupados de la base de datos para mejorar el rendimiento de las consultas.

Script:

```
CREATE DATABASE GradosTitulos
GO

USE master;
GO

-- Crear un nuevo Filegroup para índices
ALTER DATABASE GradosTitulos
ADD FILEGROUP IdxGroup;
GO

-- Agregar archivo físico al Filegroup
ALTER DATABASE GradosTitulos
ADD FILE
(
    NAME = 'IdxGroup_Data',
    FILENAME = 'C:\SQLData\IdxGroup_Data.ndf',
    SIZE = 50MB,
    FILEGROWTH = 10MB
)
TO FILEGROUP IdxGroup;
GO

-- Ejemplo de creación de índice no agrupado
CREATE NONCLUSTERED INDEX IX_Persona_DNI
ON Persona(DNI)
ON IdxGroup;
GO

-- Ejemplo de otro índice
CREATE NONCLUSTERED INDEX IX-Tramite_Matricula
ON Tramite(Matricula)
ON IdxGroup;
GO
```

Justificación:

Se crea un **Filegroup secundario llamado IdxGroup** para separar físicamente los índices no agrupados de los datos principales de la base de datos.

Esta técnica permite:

- Reducir la contención de operaciones de lectura y escritura.
- Mejorar el rendimiento de las consultas que utilizan índices.

- Facilitar tareas de mantenimiento y respaldo.
- Distribuir la carga de I/O entre diferentes archivos físicos.

Explicación de las buenas prácticas:

Separación lógica y física de los datos

Se utiliza un Filegroup independiente para índices, permitiendo una mejor organización y administración de la base de datos.

Optimización del rendimiento

Los índices se almacenan en una ubicación independiente para acelerar búsquedas, filtros y ordenamientos frecuentes.

Escalabilidad

La estructura permite agregar más índices en el futuro sin afectar la organización física de los datos principales.

Problema 2)

Enunciado:

Desarrollar una consulta que muestre el espacio total asignado, el espacio utilizado y el espacio libre de todos los archivos físicos y grupos de archivos que componen actualmente la base de datos **GradosTitulos**.

Script:

```
USE GradosTitulos;
GO

SELECT
    fg.name AS Filegroup,
    df.name AS Archivo_Logico,
    df.physical_name AS Archivo_Fisico,
    df.size * 8 / 1024 AS Tamaño_MB,
    FILEPROPERTY(df.name, 'SpaceUsed') * 8 / 1024 AS Usado_MB,
    (df.size - FILEPROPERTY(df.name, 'SpaceUsed')) * 8 / 1024 AS Libre_MB
FROM sys.database_files df
LEFT JOIN sys.filegroups fg
    ON df.data_space_id = fg.data_space_id
ORDER BY fg.name;
```

Justificación:

La consulta utiliza las vistas de catálogo del sistema:

- sys.database_files
- sys.filegroups

para obtener información precisa sobre los archivos de datos asociados a la base de datos.

Explicación:

Uso de vistas de sistema oficiales

Se emplean las vistas proporcionadas por SQL Server, garantizando información confiable y actualizada.

Monitoreo preventivo

La consulta permite detectar problemas de capacidad antes de que afecten el funcionamiento del sistema.

Conversión de páginas a megabytes

SQL Server almacena tamaños en páginas de 8 KB; se realiza la conversión a MB para facilitar la interpretación.

Organización de resultados

Se ordenan los registros por Filegroup para mejorar la lectura y el análisis administrativo.

Ejercicio 3

Enunciado:

Verificar el nivel de compatibilidad actual de la base de datos **GradosTitulos** y elevarlo a la versión más reciente disponible del motor SQL Server para habilitar optimizaciones avanzadas de consultas, como **Intelligent Query Processing**.

Script:

```
--2.1
SELECT
    name AS BaseDatos,
    compatibility_level AS NivelCompatibilidad
FROM sys.databases
WHERE name = 'GradosTitulos';
GO
```

Justificación:

El nivel de compatibilidad determina qué características y optimizaciones del motor de base de datos estarán disponibles para una base de datos específica.

Al elevar el nivel de compatibilidad se obtienen beneficios como:

- Mejor optimización de consultas.
- Uso de Intelligent Query Processing.
- Mejor estimación de cardinalidad.

- Optimización automática de planes de ejecución.
- Mejor rendimiento en consultas complejas.

La consulta inicial permite identificar el nivel actual antes de realizar cambios

Explicación:

Verificación previa

Antes de modificar configuraciones críticas se consulta el estado actual de la base de datos.

Validación posterior

Se comprueba que el cambio se haya aplicado correctamente mediante una consulta de confirmación.

Compatibilidad controlada

La actualización se realiza utilizando niveles soportados oficialmente por la versión instalada de SQL Server.

Optimización del rendimiento

Mantener un nivel de compatibilidad actualizado permite aprovechar mejoras del motor sin modificar la estructura de los datos.

EJERCICIO 4)

Enunciado:

Configurar la propiedad **PAGE_VERIFY** de la base de datos **GradosTitulos** en modo **CHECKSUM** para que SQL Server detecte posibles corrupciones de páginas de datos causadas por fallos de hardware, disco o sistema operativo

Script:

```
USE SELECT
    name AS BaseDatos,
    page_verify_option_desc AS VerificacionPagina
FROM sys.databases
WHERE name = 'GradosTitulos';
GO
```

Justificación:

La opción **CHECKSUM** hace que SQL Server genere y almacene una suma de verificación para cada página de datos cuando esta es escrita en disco.

Cuando la página es leída nuevamente, SQL Server compara el valor almacenado con uno recién calculado.

Si los valores no coinciden, el motor detecta inmediatamente una posible corrupción

física de datos. Esta configuración es considerada el método más seguro de verificación de integridad de páginas disponible en SQL Server.

Explicación:

Protección de la integridad de los datos

CHECKSUM ayuda a detectar daños físicos antes de que afecten gravemente la información almacenada.

Configuración recomendada por Microsoft

Es la opción recomendada para entornos de producción debido a su alta capacidad de detección de errores.

Verificación antes y después del cambio

Se consulta la configuración actual y posteriormente se valida que el cambio haya sido aplicado correctamente.

Mantenimiento preventivo

Permite identificar problemas de hardware o almacenamiento de forma temprana.

EJERCICIO 5)

Enunciado:

Realizar un respaldo diferencial (**DIFFERENTIAL BACKUP**) de la base de datos **GradosTitulos** para optimizar el espacio de almacenamiento y reducir el tiempo de ejecución de las copias de seguridad diarias, almacenándolo en un archivo específico.

Script:

```
SELECT
    name AS BaseDatos,
    recovery_model_desc AS ModeloRecuperacion
FROM sys.databases
WHERE name = 'GradosTitulos';
GO
BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Diferencial.bak'
WITH DIFFERENTIAL,
    INIT,
    STATS = 10;
GO
```

Justificación:

El respaldo diferencial almacena únicamente los cambios realizados desde el último respaldo completo (**FULL BACKUP**).

Sus ventajas principales son:

- Reduce el tamaño de los archivos de respaldo.
- Disminuye el tiempo requerido para generar copias de seguridad.
- Reduce el consumo de recursos del servidor.
- Simplifica la restauración al requerir únicamente el respaldo completo y el último diferencial.

Esta estrategia es adecuada para tareas de respaldo diario

Explicación:

Estrategia de respaldo escalonada

Se combina un respaldo completo inicial con respaldos diferenciales periódicos para optimizar recursos.

Uso de INIT

Garantiza que el archivo de respaldo se sobrescriba evitando acumulación innecesaria de copias antiguas.

Validación posterior

La consulta RESTORE HEADERONLY permite verificar que el respaldo fue creado correctamente.

EJERCICIO 6)

Enunciado:

Realizar un respaldo del archivo de registro de transacciones (**Transaction Log Backup**) de la base de datos **GradosTitulos** bajo el modelo de recuperación **FULL**, con el fin de liberar espacio reutilizable en el archivo .ldf y permitir recuperaciones hasta un punto específico en el tiempo.

Script:

```

ALTER DATABASE GradosTitulos
SET RECOVERY FULL;
GO
BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Full.bak'
WITH INIT,
    STATS = 10;
GO
BACKUP LOG GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Log.trn'
WITH INIT,
    STATS = 10;
GO

```

Justificación:

Los respaldos de Log solo pueden ejecutarse cuando la base de datos utiliza los modelos de recuperación **FULL** o **BULK_LOGGED**.

El respaldo del Log registra todas las transacciones realizadas desde el último respaldo del registro, permitiendo:

- Recuperación hasta un punto exacto en el tiempo.
- Minimización de pérdida de datos.
- Liberación del espacio reutilizable del archivo de transacciones.
- Mayor protección ante fallos del sistema.

Además, SQL Server requiere un respaldo completo

Explicación:

Uso del modelo FULL

Permite implementar estrategias avanzadas de recuperación ante desastres.

Respaldo completo previo

Se genera un respaldo FULL antes del primer respaldo LOG para iniciar correctamente la cadena de recuperación.

Separación de archivos de respaldo

Los respaldos FULL y LOG se almacenan en archivos independientes para facilitar la administración.

EJERCICIO 7)

Enunciado:

Crear un rol personalizado denominado **RolSecretariaAcademica** dentro de la base de datos **GradosTitulos**. El rol debe permitir la consulta de información (**SELECT**) en el esquema **Academico** y en el esquema **Tramite**, además de autorizar la inserción de registros (**INSERT**) únicamente en la tabla **Tramite.Tramite**.

Script:

```
USE GradosTitulos;
GO

-- Crear el rol personalizado
CREATE ROLE RolSecretariaAcademica;
GO

-- Permisos de lectura sobre el esquema Academico
GRANT SELECT
ON SCHEMA::Academico
TO RolSecretariaAcademica;
GO

-- Permisos de lectura sobre el esquema Tramite
GRANT SELECT
ON SCHEMA::Tramite
TO RolSecretariaAcademica;
GO

-- Permiso de inserción únicamente sobre la tabla Tramite
GRANT INSERT
ON Tramite.Tramite
TO RolSecretariaAcademica;
GO

-- Ejemplo de asignación del rol a un usuario
```


Justificación:

La creación de un rol personalizado permite agrupar permisos relacionados con las funciones de la secretaría académica sin asignarlos directamente a cada usuario.

Los permisos otorgados cumplen el principio de mínimo privilegio:

- **SELECT** sobre los esquemas **Academico** y **Tramite** para consultar información.
- **INSERT** únicamente sobre la tabla **Tramite.Tramite** para registrar nuevos trámites.
- No se conceden permisos de modificación (**UPDATE**) ni eliminación (**DELETE**) para proteger la integridad de los datos.

Explicación:

Uso de roles personalizados

Permite administrar permisos de forma centralizada y reutilizable.

Aplicación del principio de mínimo privilegio

Los usuarios reciben únicamente los permisos necesarios para realizar sus funciones.

Asignación de permisos por esquema

Otorgar permisos sobre esquemas simplifica la administración cuando existen múltiples tablas relacionadas.

Separación de responsabilidades

Las operaciones de consulta y registro se controlan de manera independiente.

EJERCICIO 8)

Enunciado:

Restringir explícitamente al usuario **OperadorTramite** cualquier acceso de lectura sobre la tabla **Tramite.Pago**, asegurando que la denegación prevalezca sobre cualquier permiso otorgado por roles o asignaciones previas.

Script:

```
USE GradosTitulos;
GO

-- Denegar lectura sobre la tabla Tramite.Pago
DENY SELECT
ON Tramite.Pago
TO OperadorTramite;
GO

-- Verificar permisos aplicados
SELECT
    dp.state_desc,
    dp.permission_name,
    OBJECT_NAME(dp.major_id) AS Objeto
FROM sys.database_permissions dp
WHERE dp.grantee_principal_id =
    USER_ID('OperadorTramite');
```

Justificación:

La instrucción **DENY** bloquea explícitamente un permiso para un usuario determinado.

En SQL Server, un permiso denegado tiene prioridad sobre los permisos concedidos mediante:

- Roles de base de datos.
- Roles personalizados.
- Permisos otorgados directamente mediante **GRANT**.

Por ello, aunque el usuario pertenezca a un rol con permisos de lectura sobre la tabla **Pago**, la cláusula **DENY SELECT** impedirá el acceso a dicha información.

Explicación:

Protección de información crítica

Los datos financieros suelen requerir niveles adicionales de seguridad.

Uso de DENY para restricciones específicas

Permite implementar excepciones de seguridad sin modificar los permisos generales de otros roles.

Control granular de acceso

La restricción se aplica únicamente a la tabla necesaria, evitando afectar otras operaciones del usuario.

Auditoría y verificación

Se incluye una consulta para validar los permisos asignados y facilitar futuras revisiones.

EJERCICIO 9)

Enunciado:

Otorgar al usuario **PracticanteSistemas** permisos de lectura (**SELECT**) únicamente sobre las columnas **IdPersona**, **Nombres**, **ApellidoPaterno** y **Correo** de la tabla **Academico.Persona**, restringiendo el acceso al resto de información almacenada en dicha tabla.

Script:

```

USE GradosTitulos;
GO

-- Otorgar permisos SELECT únicamente sobre columnas específicas
GRANT SELECT
(
    IdPersona,
    Nombres,
    ApellidoPaterno,
    Correo
)
ON OBJECT::Academico.Persona
TO PracticanteSistemas;
GO

-- Verificar permisos asignados
SELECT
    dp.permission_name,
    dp.state_desc,
    OBJECT_NAME(dp.major_id) AS Objeto
FROM sys.database_permissions dp
WHERE dp.grantee_principal_id =
    USER_ID('PracticanteSistemas');
GO

```

Justificación:

Los permisos a nivel de columna permiten controlar de forma precisa qué información puede consultar un usuario dentro de una misma tabla.

En este caso, el usuario **PracticanteSistemas** requiere acceder únicamente a datos de contacto de los docentes:

- IdPersona
- Nombres
- ApellidoPaterno
- Correo

Sin embargo, otras columnas podrían contener información sensible como:

- DNI
- Teléfono
- Dirección
- Fecha de nacimiento
- Información administrativa

Por ello se limita el acceso exclusivamente a las columnas necesarias para el desempeño de sus funciones.

Explicación:

Aplicación del principio de mínimo privilegio

El usuario recibe únicamente los permisos indispensables para realizar sus actividades.

Protección de datos sensibles

La información personal y confidencial permanece restringida.

Seguridad granular

Los permisos se asignan directamente a columnas específicas en lugar de a la tabla completa.

Cumplimiento normativo

Ayuda a cumplir políticas institucionales de privacidad y protección de datos.

Reducción de riesgos

Disminuye la exposición accidental o intencional de información confidencial.

EJERCICIO 10)

Enunciado:

Conceder al usuario **OperadorTramite** permisos exclusivos para ejecutar el procedimiento almacenado **Tramite.SP_RegistrarTramite**, permitiéndole registrar nuevas solicitudes sin otorgarle acceso directo de escritura sobre las tablas involucradas.

Script:

```
USE GradosTitulos;
GO

-- Otorgar permiso de ejecución sobre el procedimiento
GRANT EXECUTE
ON OBJECT::Tramite.SP_RegistrarTramite
TO OperadorTramite;
GO

-- Verificar permisos otorgados
SELECT
    dp.permission_name,
    dp.state_desc,
    OBJECT_NAME(dp.major_id) AS Objeto
FROM sys.database_permissions dp
WHERE dp.grantee_principal_id =
    USER_ID('OperadorTramite');
GO
```

Justificación:

El permiso **EXECUTE** permite que un usuario ejecute un procedimiento almacenado sin necesidad de poseer permisos directos sobre las tablas utilizadas internamente.

Esta estrategia ofrece varias ventajas:

- Control centralizado de la lógica de negocio.
- Mayor seguridad sobre los datos.
- Restricción de accesos directos a tablas críticas.
- Validación de información antes de realizar inserciones o modificaciones.
- Facilidad para auditar operaciones realizadas por los usuarios.

El usuario únicamente puede ejecutar el procedimiento autorizado, mientras que la manipulación de los datos queda controlada por la lógica implementada en dicho procedimiento.

Explicación:

Uso de procedimientos almacenados

Centralizan la lógica de negocio dentro de la base de datos.

Seguridad basada en ejecución

Los usuarios interactúan con procedimientos en lugar de acceder directamente a las tablas.

Menor superficie de ataque

Se reducen los riesgos asociados a modificaciones accidentales o maliciosas.

Mantenimiento simplificado

Los cambios en la lógica del proceso se realizan únicamente dentro del procedimiento almacenado.

Control y auditoría

Las operaciones pueden ser monitoreadas y registradas de manera más eficiente.

EJERCICIO 11)

Enunciado:

Desarrollar una consulta en T-SQL que analice los planes de ejecución almacenados en la memoria caché del servidor y muestre las 5 consultas que han consumido la mayor cantidad de tiempo de CPU en la base de datos **GradosTitulos**.

Script:

```

USE GradosTitulos;
GO

SELECT TOP 5
    qs.total_worker_time / 1000 AS CPU_Total_ms,
    qs.execution_count AS Veces_Ejecutada,
    (qs.total_worker_time / qs.execution_count) / 1000 AS CPU_Promedio_ms,
    qs.total_elapsed_time / 1000 AS Tiempo_Total_ms,
    DB_NAME(st.dbid) AS BaseDatos,
    SUBSTRING(
        st.text,
        (qs.statement_start_offset / 2) + 1,
        (
            CASE qs.statement_end_offset
                WHEN -1 THEN DATALENGTH(st.text)
                ELSE qs.statement_end_offset
            END
            - qs.statement_start_offset
        ) / 2
    ) + 1
    ) AS ConsultaSQL
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
WHERE DB_NAME(st.dbid) = 'GradosTitulos'
ORDER BY qs.total_worker_time DESC;
GO

```

Justificación:

La consulta utiliza la DMV:

(sys.dm_exec_query_stats)

para obtener estadísticas acumuladas de ejecución de consultas almacenadas en caché.

Los indicadores más importantes analizados son:

- **total_worker_time** → tiempo total de CPU consumido.
- **execution_count** → cantidad de ejecuciones.
- **total_elapsed_time** → tiempo total de ejecución.
- **sql_handle** → identificador de la consulta almacenada.

La combinación con:

Explicación:

Uso de DMVs oficiales

Se emplean vistas de administración dinámica proporcionadas por SQL Server para obtener información confiable.

Análisis basado en métricas reales

Las decisiones de optimización se fundamentan en datos de ejecución y no en suposiciones.

Priorización de problemas

Se muestran únicamente las cinco consultas más costosas para enfocar los esfuerzos de optimización.

Monitoreo proactivo

Permite detectar consultas ineficientes antes de que afecten significativamente el rendimiento.

Diagnóstico de rendimiento

Facilita la identificación de procedimientos almacenados o consultas que requieren ajuste de índices o reescritura.

EJERCICIO 12)

Enunciado:

Implementar una consulta analítica que permita verificar el uso real de los índices no agrupados creados en la base de datos **GradosTitulos**, identificando aquellos que presentan poco o ningún uso en búsquedas pero continúan consumiendo espacio y recursos de mantenimiento.

Script:

```

SELECT
    OBJECT_NAME(i.object_id) AS Tabla,
    i.name AS Indice,
    i.type_desc AS TipoIndice,
    ISNULL(us.user_seeks,0) AS Busquedas,
    ISNULL(us.user_scans,0) AS Escaneos,
    ISNULL(us.user_lookups,0) AS Lookups,
    ISNULL(us.user_updates,0) AS Actualizaciones,
    us.last_user_seek AS UltimaBusqueda
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats us
    ON i.object_id = us.object_id
    AND i.index_id = us.index_id
    AND us.database_id = DB_ID()
WHERE i.type_desc = 'NONCLUSTERED'
    AND i.name IS NOT NULL
ORDER BY Busquedas ASC, Actualizaciones DESC;
GO

```

	Tabla	Indice	TipoIndice	Busquedas	Escaneos	Lookups	Actualizaciones
1	Facultad	UQ_Facultad_Nombre	NONCLUSTERED	0	0	0	2
2	EstadoTramite	UQ_EstadoTramite_Descripcion	NONCLUSTERED	0	0	0	2
3	TipoTramite	UQ_TipoTramite_Descripcion	NONCLUSTERED	0	0	0	2
4	Sustentacion	UQ_Sust_Trabajo_Intento	NONCLUSTERED	0	0	0	0
5	Sustentacion	IX_Sust_Trabajo	NONCLUSTERED	0	0	0	0
6	LineaInvestigacion	UQ_LineaInvestigacion_Nombre	NONCLUSTERED	0	0	0	0
7	Resolucion	UQ_Resolucion_Nro	NONCLUSTERED	0	0	0	0
8	Resolucion	IX_Res_Tramite	NONCLUSTERED	0	0	0	0

Justificación:

La solución utiliza la DMV:

(sys.dm_db_index_usage_stats)

que almacena estadísticas de uso de los índices desde el último reinicio del servicio SQL Server.

Se analizan los siguientes indicadores:

- **user_seeks** → búsquedas eficientes utilizando el índice.
- **user_scans** → escaneos completos realizados sobre el índice.
- **user_lookups** → búsquedas complementarias.
- **user_updates** → operaciones de inserción, actualización o eliminación que obligan a mantener el índice.

Un índice con:

Busquedas = 0

Actualizaciones > 0

puede ser candidato para revisión o eliminación, ya que consume recursos sin aportar beneficios significativos al rendimiento.

Explicación:

Monitoreo continuo de índices

Permite evaluar si los índices realmente están siendo utilizados por las consultas.

Identificación de índices innecesarios

Ayuda a detectar índices que consumen almacenamiento y tiempo de mantenimiento sin aportar mejoras.

Optimización del rendimiento

La eliminación o ajuste de índices poco utilizados reduce la carga de escritura sobre las tablas.

Uso de métricas históricas

Las estadísticas permiten tomar decisiones basadas en evidencia y no en estimaciones.

Administración eficiente de recursos

Contribuye a mantener una estructura de índices equilibrada entre rendimiento de lectura y costo de mantenimiento.

EJERCICIO 13)

Enunciado:

Configurar un operador denominado **DBA_Alertas** en SQL Server Agent para que reciba notificaciones automáticas por correo electrónico cuando falle el Job encargado del respaldo diario de la base de datos **GradosTitulos**.

Script:

```

--Operador
USE msdb;
GO

EXEC sp_add_operator
    @name = 'DBA_Alertas',
    @enabled = 1,
    @email_address = 'dbaalertas@universidad.edu.pe';
GO

--asociar JOB
USE msdb;
GO

EXEC sp_update_job
    @job_name = 'Backup_Diario_GradosTitulos',
    @notify_level_email = 2,
    @notify_email_operator_name = 'DBA_Alertas';
GO

USE msdb;
GO
EXEC sp_help_operator;
GO

```

	id	name	enabled	email_address	last_email_date	last_email_time	pager_address	last
1	1	DBA_Alertas	1	dbaalertas@universidad.edu.pe	0	0	NULL	0

Justificación:

La solución utiliza los procedimientos almacenados del sistema SQL Server Agent:

- sp_add_operator
- sp_update_job

El operador representa una persona o grupo responsable de administrar el servidor.

La opción:

@notify_level_email = 2

indica que SQL Server enviará una notificación únicamente cuando el Job falle.

De esta manera los administradores pueden reaccionar rápidamente ante errores en los procesos de respaldo, reduciendo el riesgo de pérdida de información.

Explicación:

Monitoreo automatizado

La supervisión se realiza automáticamente sin intervención manual.

Respuesta temprana ante incidentes

Los administradores son informados inmediatamente cuando ocurre un error.

Uso de SQL Server Agent

Se aprovechan las herramientas nativas de administración del motor.

Prevención de pérdida de datos

Permite detectar fallos en respaldos antes de que comprometan la estrategia de recuperación.

Administración centralizada

Las alertas pueden dirigirse a un equipo completo mediante una dirección de correo institucional.

EJERCICIO 14)

Enunciado:

Desarrollar un script automatizado que identifique índices de la base de datos **GradosTitulos** con un nivel de fragmentación superior al 30% y ejecute una reconstrucción (**ALTER INDEX REBUILD**) para mejorar el rendimiento de las consultas.

Script:

```
USE GradosTitulos;
GO

)SELECT
    OBJECT_NAME(ps.object_id) AS Tabla,
    i.name AS Indice,
    ps.avg_fragmentation_in_percent AS Fragmentacion
FROM sys.dm_db_index_physical_stats
(
    DB_ID(),
    NULL,
    NULL,
    NULL,
    'LIMITED'
) ps
INNER JOIN sys.indexes i
    ON ps.object_id = i.object_id
    AND ps.index_id = i.index_id
WHERE ps.avg_fragmentation_in_percent > 30
ORDER BY ps.avg_fragmentation_in_percent DESC;
GO
```

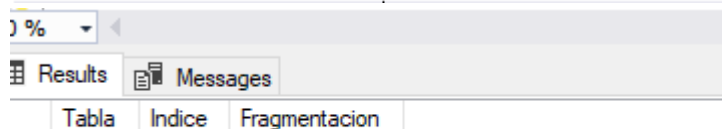


Tabla	Indice	Fragmentacion
-------	--------	---------------

Justificación:

La DMV:

`sys.dm_db_index_physical_stats`

permite medir el nivel de fragmentación física de los índices.

Cuando la fragmentación supera aproximadamente el 30%, Microsoft recomienda realizar una reconstrucción mediante:

`ALTER INDEX REBUILD`

porque:

- Reorganiza completamente las páginas del índice.
- Actualiza estadísticas.
- Reduce lecturas innecesarias.
- Mejora el rendimiento de búsquedas y ordenamientos.

La generación dinámica del código permite reconstruir todos los índices afectados sin necesidad de hacerlo manualmente.

Explicación:

Mantenimiento preventivo

La fragmentación se corrige antes de que impacte significativamente el rendimiento.

Automatización de tareas repetitivas

Reduce errores humanos y tiempo administrativo.

Uso de umbrales recomendados

Se utiliza el criterio de fragmentación superior al 30%, ampliamente recomendado en SQL Server.

Optimización de consultas

Los índices reconstruidos permiten accesos más eficientes a los datos.

Programación mediante SQL Server Agent

La tarea puede ejecutarse periódicamente sin intervención manual, garantizando un mantenimiento constante de la base de datos.

EJERCICIO 15)

Enunciado:

Modificar la tabla **Tramite.Pago** agregando una restricción **CHECK** que garantice que el valor del campo **Monto** siempre sea mayor que cero, evitando el registro de pagos nulos o negativos dentro del sistema.

Script:

```

USE GradosTitulos;
GO

ALTER TABLE Tramite.Pago
ADD CONSTRAINT CK_Pago_Monto_Positivo
CHECK (Monto > 0);
GO

-- Verificar la restricción creada
SELECT
    name AS Restriccion,
    type_desc AS Tipo
FROM sys.objects
WHERE type = 'C'
AND name = 'CK_Pago_Monto_Positivo';
GO

```

Results		Messages	
	Restriccion	Tipo	
1	CK_Pago_Monto_Positivo	CHECK_CONSTRAINT	

Justificación:

La restricción **CHECK** permite implementar reglas de negocio directamente en la base de datos.

En este caso:

$\text{Monto} > 0$

garantiza que ningún pago registrado tenga un importe igual o menor a cero.

Esto evita inconsistencias en la información financiera y asegura la calidad de los datos almacenados.

Explicación:

Validación a nivel de base de datos

Las reglas de negocio se aplican independientemente de la aplicación que inserte los datos.

Protección de la integridad de los datos

Se impide el almacenamiento de valores inválidos.

Uso de nombres descriptivos

La restricción posee un nombre identificable para facilitar mantenimiento y auditoría.

Prevención temprana de errores

Los datos incorrectos son rechazados antes de ser almacenados.

Consistencia institucional

Se garantiza que todos los pagos registrados sean válidos.

EJERCICIO 16)

Enunciado:

Modificar la columna **NroRecibo** de la tabla **Tramite.Pago**, ampliando su longitud desde **VARCHAR(30)** hasta **VARCHAR(60)**, manteniendo la propiedad de permitir valores nulos (**NULL**).

Script:

```
USE GradosTitulos;
GO

ALTER TABLE Tramite.Pago
ALTER COLUMN NroRecibo VARCHAR(60) NULL;
GO

-- Verificar modificación
SELECT
    COLUMN_NAME,
    DATA_TYPE,
    CHARACTER_MAXIMUM_LENGTH,
    IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'Pago'
AND COLUMN_NAME = 'NroRecibo';
GO
```

Results

Messages

	COLUMN_NAME	DATA_TYPE	CHARACTER_MAXIMUM_LENGTH	IS_NULLABLE
1	NroRecibo	varchar	60	YES

Justificación:

El crecimiento de los requerimientos funcionales puede demandar almacenar códigos o números de recibo más extensos.

La instrucción:

ALTER COLUMN

permite ampliar la capacidad de almacenamiento sin afectar los datos existentes.

Además, se conserva la característica:

NULL

para mantener la compatibilidad con registros donde dicho dato aún no se encuentre disponible.

Explicación:

Evolución controlada del esquema

La modificación se realiza sin eliminar ni recrear la tabla.

Conservación de datos existentes

No se pierde información almacenada previamente.

Compatibilidad con versiones anteriores

Se mantiene la capacidad de almacenar valores nulos.

Verificación posterior

Se consulta el diccionario de datos para confirmar el cambio.

Escalabilidad

La estructura queda preparada para futuros requerimientos.

EJERCICIO 17)

Enunciado:

Crear un trigger denominado **TR_Auditoria_ModificarPago** sobre la tabla **Tramite.Pago** que registre automáticamente las modificaciones realizadas al monto de un pago, almacenando el valor anterior, el nuevo valor, el usuario responsable y la fecha del cambio.

Script:

Justificación:

Los triggers **AFTER UPDATE** se ejecutan automáticamente después de una modificación exitosa.

Las tablas virtuales:

- **deleted** → contienen los valores anteriores.
- **inserted** → contienen los nuevos valores.

Esto permite almacenar evidencia completa de cada modificación realizada sobre los pagos.

La auditoría obtenida resulta útil para:

- Seguimiento de cambios.
- Detección de errores.
- Cumplimiento normativo.
- Investigación forense de incidentes.

Explicación:

Trazabilidad completa

Cada modificación queda registrada con usuario y fecha.

Auditoría automática

No depende de acciones manuales de los usuarios.

Uso de SYSTEM_USER

Permite identificar quién realizó la modificación.

Registro histórico permanente

La información auditada permanece disponible para consultas futuras.

Seguridad institucional

Facilita investigaciones ante modificaciones indebidas.

EJERCICIO 18)

Enunciado:

Implementar un trigger de tipo **INSTEAD OF** sobre la tabla **Academico.Docente** que impida realizar operaciones de eliminación o modificación durante fines de semana (sábado y domingo), cancelando automáticamente la transacción.

Script:


```

USE GradosTitulos;
GO

CREATE TRIGGER TR_Seguridad_Docente_Update
ON Academico.Docente
INSTEAD OF UPDATE
AS
BEGIN
    IF DATENAME(WEEKDAY, GETDATE()) IN ('Saturday', 'Sunday')
    BEGIN
        RAISERROR(
            'No se permiten modificaciones durante fines de semana.',
            16,
            1
        );
        RETURN;
    END
END;

UPDATE d
SET
    d.Nombres = i.Nombres,
    d.ApellidoPaterno = i.ApellidoPaterno
FROM Academico.Docente d
INNER JOIN inserted i
    ON d.IdDocente = i.IdDocente;

END;
GO

```

Justificación:

Los triggers **INSTEAD OF** interceptan la operación antes de que sea ejecutada.

El sistema verifica el día actual mediante:

DATENAME(WEEKDAY, GETDATE())

Si la operación ocurre un sábado o domingo:

- Se genera un error.
- La operación es cancelada.
- No se modifica la información.

Esto implementa una política de seguridad institucional directamente en la base de datos.

Explicación:

Seguridad centralizada

Las restricciones se aplican desde la base de datos y no desde la aplicación.

Prevención de cambios no autorizados

Se bloquean modificaciones durante períodos restringidos.

Uso de INSTEAD OF

Permite interceptar operaciones antes de ejecutarse.

Protección de información académica

Evita alteraciones fuera de los horarios establecidos.

EJERCICIO 19)

Enunciado:

Ante la eliminación accidental de registros de la tabla **Catalogo.EstadoTramite**, recuperar la información sin restaurar completamente la base de datos, utilizando consultas entre bases de datos (**Cross-Database Queries**) para copiar los registros desde una base auxiliar denominada **GradosTitulos_Auxiliar** hacia la base principal **GradosTitulos**

Script:

Justificación:

La recuperación parcial permite restaurar únicamente la información afectada sin necesidad de detener el sistema ni restaurar la base de datos completa.

Las consultas **Cross-Database** permiten acceder simultáneamente a:

- Base principal: **GradosTitulos**
- Base de respaldo auxiliar: **GradosTitulos_Auxiliar**

La cláusula:

WHERE NOT EXISTS

evita insertar registros duplicados durante el proceso de recuperación.

Este método reduce considerablemente el tiempo de recuperación y minimiza el impacto operativo sobre los usuarios.

Explicación:

Recuperación selectiva

Se restauran únicamente los datos afectados.

Minimización del tiempo de inactividad

No es necesario restaurar la base de datos completa.

Prevención de duplicados

Se valida previamente la existencia de registros.

Verificación posterior

Se comprueba que la información haya sido recuperada correctamente.

EJERCICIO 20)

Enunciado:

Después de una restauración de emergencia, ejecutar el comando **DBCC CHECKDB** sobre la base de datos **GradosTitulos** para validar la integridad física y lógica de las páginas de datos, índices y estructuras internas, garantizando que no existan errores de corrupción.

Script:

Justificación:

DBCC CHECKDB es la herramienta oficial de SQL Server para comprobar la integridad de una base de datos.

Durante la ejecución realiza validaciones sobre:

- Estructuras físicas de almacenamiento.
- Páginas de datos.
- Índices.
- Relaciones internas.
- Consistencia lógica de tablas.

Si existen problemas de corrupción, el comando genera mensajes detallados indicando el objeto afectado y el tipo de error detectado.

Por ello es una práctica obligatoria después de procesos de restauración o recuperación de desastres.

Explicación:

Validación posterior a la restauración

Toda recuperación debe verificarse antes de poner la base nuevamente en producción.

Uso de herramientas oficiales

DBCC CHECKDB es el mecanismo recomendado por Microsoft para auditorías de integridad.

Detección temprana de corrupción

Permite identificar errores antes de que afecten las operaciones del negocio.

Mantenimiento preventivo

La ejecución periódica ayuda a monitorear el estado de la base de datos.